

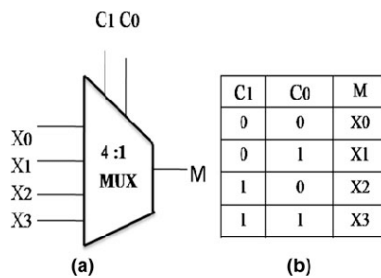
W3_hw2 Report

[1]Design and Implement 4:1 MUX and 1:4 DEMUX

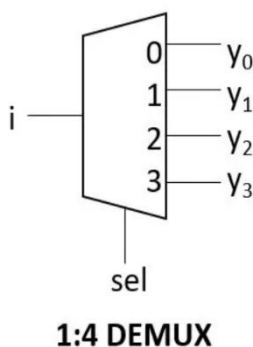
1. Objectives:

- (1) 주어진 2:1 MUX와 1:2 DEMUX의 작동 원리를 파악하여 이해한 내용을 바탕으로 4:1 MUX와 1:4 DEMUX의 작동 원리를 파악한다.
- (2) 4:1 MUX와 1:4 DEMUX를 Dataflow, Gate-Level Modeling 방법을 사용하여 source code를 구현하고 실행을 확인한다.

2. Theoretical Approach:



4:1 MUX의 Diagram과 Truth Table은 왼쪽과 같다. Inputs를 $X_0 \sim X_3$ 까지 입력하면 select bits C_1, C_0 에 따라 output이 출력된다. 이에 대한 진리표가 왼쪽 그림의 b이다. Select bits를 s_1, s_0 , inputs를 $I_0 \sim I_3$, output을 Y 라고 하면 Boolean expression은 $Y = s_1 s_0 I_3 + s_1 \bar{s}_0 I_2 + \bar{s}_1 s_0 I_1 + \bar{s}_1 \bar{s}_0 I_0$ 이다.



Truth Table

sel[0]	sel[1]	Y ₀	Y ₁	Y ₂	Y ₃
0	0	i	0	0	0
0	1	0	i	0	0
1	0	0	0	i	0
1	1	0	0	0	i

1:4 DEMUX의 Diagram과 Truth Table은 왼쪽의 그림과 같다. Input에 I 를 넣고 select bits에 sel_0, sel_1 을 입력하면 select bits 값에 따라 outputs가 $y_0 \sim y_3$ 로 도출된다. 진리표를 이용해 1:4 DEMUX의 Boolean expression을 나타내면 $Y_3 = s_1 s_0 I$, $Y_2 = s_1 s_0' I$, $Y_1 = s_1' s_0 I$, $Y_0 = s_1' s_0' I$

로 표현할 수 있다.

3. Verilog Implementation:

(1) Dataflow Modeling of 4:1 MUX :

```
module four_to_one_mux_dataflow_module (a, b, c, d, s0, s1, out);
```

```
    input a, b, c, d;
```

```

input s0, s1;

output out;

assign out = (!s1 && !s0 && a) || (!s1 && s0 && b) || (s1 && !s0 && c) || (s1 && s0 && d);

endmodule

```

4:1 MUX의 Boolean expression인 $Y = s_1 s_0 I_3 + s_1 \bar{s}_0 I_2 + \bar{s}_1 s_0 I_1 + \bar{s}_1 \bar{s}_0 I_0$ 을 이용하였다. input으로 a, b, c, d, s0, s1을 선언하였으므로 Boolean expression에 Logical Operators로 입력하여 out을 할당하였다.

(2) Gate-Level Modeling of 4:1 MUX :

```

module four_to_one_mux_gatelevel_module (a, b, c, d, s0, s1, out);

    input a, b, c, d;

    input s0, s1;

    output out;

    wire not_1_output, not_2_output;

    wire and_1_output, and_2_output, and_3_output, and_4_output,
        and_5_output, and_6_output, and_7_output, and_8_output;

    wire or_1_output, or_2_output;

    not_gate not_1 (.a(s1), .out(not_1_output));

    not_gate not_2 (.a(s0), .out(not_2_output));

    and_gate and_1 (.a(not_1_output), .b(not_2_output), .out(and_1_output));

    and_gate and_2 (.a(not_1_output), .b(s0), .out(and_2_output));

    and_gate and_3 (.a(s1), .b(not_2_output), .out(and_3_output));

    and_gate and_4 (.a(s1), .b(s0), .out(and_4_output));

    and_gate and_5 (.a(and_1_output), .b(a), .out(and_5_output));

    and_gate and_6 (.a(and_2_output), .b(b), .out(and_6_output));

    and_gate and_7 (.a(and_3_output), .b(c), .out(and_7_output));

```

```

and_gate and_8 (.a(and_4_output), .b(d), .out(and_8_output));

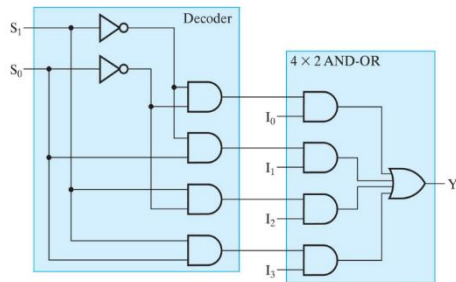
or_gate or_1 (.a(and_5_output), .b(and_6_output), .out(or_1_output));

or_gate or_2 (.a(and_7_output), .b(and_8_output), .out(or_2_output));

or_gate or_3 (.a(or_1_output), .b(or_2_output), .out(out));

```

endmodule



wire로 and gate의 outputs를 8개, or gate의 outputs를 2개 선언했으므로 왼쪽의 Diagram과 같이 gates를 설계하고 마지막 or gate만 변형했다. 그림의 I0, I1의 and gates를 or_1으로, I2, I3의 and gates를 or_2로 묶고 그 둘을 or_3로 마무리했다. 이에 대한 gate-level 모델링이 위의 소스코드에 반영되었다.

(3) Dataflow Modeling of 1:4 DEMUX :

```

module one_to_four_demux_dataflow_module (a, s0, s1, out1, out2, out3, out4);

```

```

    input a;

```

```

    input s0, s1;

```

```

    output out1, out2, out3, out4;

```

```

    assign out1 = a && !s1 && !s0;

```

```

    assign out2 = a && !s1 && s0;

```

```

    assign out3 = a && s1 && !s0;

```

```

    assign out4 = a && s1 && s0;

```

endmodule

1:4 DEMUX의 Boolean expression인 $Y_3 = s_1s_0I$, $Y_2 = s_1s_0'I$, $Y_1 = s_1's_0I$, $Y_0 = s_1's_0'I$ 을 이용하였다. input으로 a, s0, s1을 선언하였으므로 이를 Logical Operators를 이용해 assign으로 out1 ~ out4를 할당하였다.

(4) Gate-Level Modeling of 1:4 DEMUX :

```

module one_to_four_demux_gatelevel_module (a, s0, s1, out1, out2, out3, out4);

```

```

    input a;

```

```

    input s0, s1;

```

```

output out1, out2, out3, out4;

wire not_1_output, not_2_output;

wire and_1_output, and_3_output, and_5_output, and_7_output;

not_gate not_1 (.a(s1), .out(not_1_output));

not_gate not_2 (.a(s0), .out(not_2_output));

and_gate and_1 (.a(not_1_output), .b(not_2_output), .out(and_1_output));

and_gate and_3 (.a(not_1_output), .b(s0), .out(and_3_output));

and_gate and_5 (.a(s1), .b(not_2_output), .out(and_5_output));

and_gate and_7 (.a(s1), .b(s0), .out(and_7_output));

and_gate and_8 (.a(and_1_output), .b(a), .out(out1));

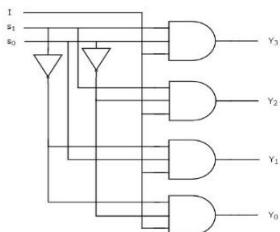
and_gate and_9 (.a(and_3_output), .b(a), .out(out2));

and_gate and_10 (.a(and_5_output), .b(a), .out(out3));

and_gate and_11 (.a(and_7_output), .b(a), .out(out4));

```

endmodule



wire로 and gate의 outputs를 4개, not gate의 outputs를 2개 선언했으므로 왼쪽의 Diagram과 같이 gates를 디자인할 수 있다. wire로 and gate outputs가 4개 선언되어 있으므로 select bits들을 우선 and gates로 묶고 그 결과를 and gate로 묶어서 outputs를 도출하는 방법으로 소스코드를 구현했다.

(5) Testbench of 4:1 MUX and 1:4 DEMUX :

```
#90 //delay to border the test of 4:1 MUX
```

```
for (count = 0; count <64; count = count +1)
```

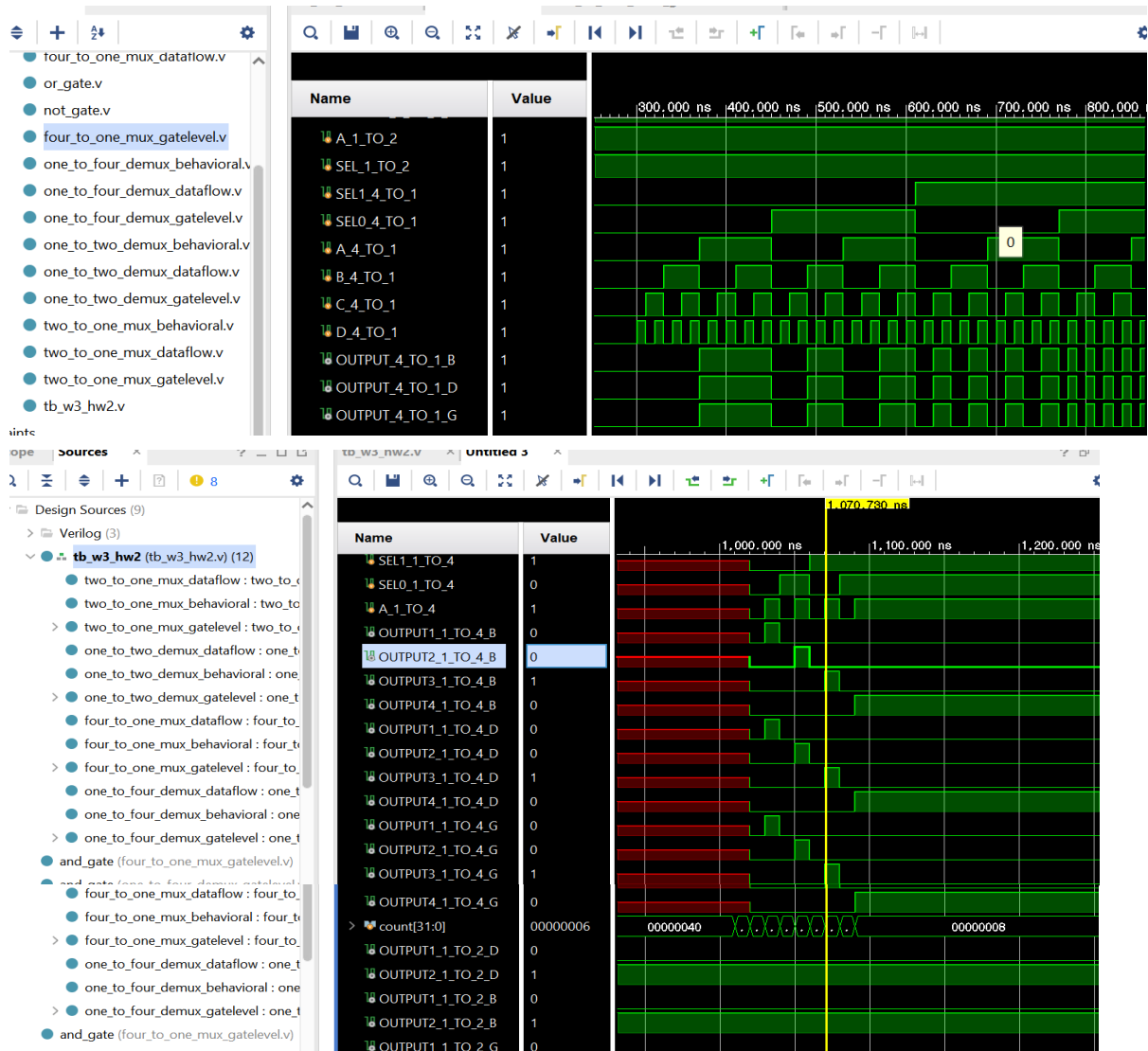
```
#10 {SEL1_4_TO_1, SEL0_4_TO_1, A_4_TO_1, B_4_TO_1, C_4_TO_1, D_4_TO_1} = count;
```

```
#90 //delay to border the test of 1:4 DEMUX
```

```
for (count = 0; count <8; count = count +1)
```

```
#10 {SEL1_1_TO_4, SEL0_1_TO_4, A_1_TO_4} = count;
```

4. Simulation Results:



4:1 MUX 의 simulation 결과 s1, s0 에 따라 나오는 output 이 Truth Table 을 정확히 반영하고 있음을 알 수 있다. 세 가지 모델링 방법에 의한 outputs 가 모두 동일한 것으로 볼 때 source code 가 틀리지 않았음을 확인할 수 있다. 1:4 DEMUX 또한 SEL1 과 SEL0 에 따라 Truth Table 의 내용을 정확히 반영하고 있음을 알 수 있다. Input 에 따라 out1 ~ out4 가 각각 출력되고 있음을 시뮬레이션을 통해 확인할 수 있다.

5. Conclusion:

4:1 MUX와 1:4 DEMUX를 이미 주어진 Behavioral Modeling 방법 외에 Dataflow, Gate-Level Modeling 방법을 이용하여 source code를 작성하고 testbench를 통해 직접 simulation 해보면서 주어진 input 값들에 따른 output(s)의 변화를 확인할 수 있었다. Select bits와 input(s)에 따라 output이 Boolean Expression에 기반한 Truth Table을 반영하고 있는 모습을 simulation을 통해 확인할 수 있었다.